Code Defense

Line by line, and the screen questions

14 July 2026

Contents

1 Part 5 — Code Defense: line-by-line mapping and screen questions

Companion to `src/pso_wifi_placement.py`. This document (a) maps concrete code to the theory of Part 1 / Part 3, and (b) pre-loads sharp answers to the questions an examiner asks while pointing at the screen.

1.1 5.1 Code → theory mapping

Code (in `src/pso_wifi_placement.py`)	Theoretical principle	Where discussed
`X = rng.uniform(p.lower, p.upper, (P, D))` (swarm init)	Population-based sampling: cover the search space, not a single start point	Part 1 §1
`V = w*V + c1*r1*(pbest-X) + c2*r2*(gbest-X)`	The PSO velocity update; `cognitive` term = personal memory (exploitation of own best), `social` term = swarm attraction (information sharing)	Part 1 (PSO); slide ”How a particle moves”
`w = w_max - (w_max-w_min)*t/(T-1)`	Exploration→exploitation schedule: high inertia early (roam), low inertia late (refine)	Part 1 (explore/exploit trade-off); Part 3 Q
`r1, r2 = rng.random(...)` fresh each step	Stochasticity that keeps the search from collapsing deterministically; the ”random wing-flutter”	Part 1 (diversity)
`V = np.clip(V, -v_max, v_max)`	Velocity clamping $V_{\max}$: stability guarantee against swarm explosion	Part 1 (PSO pathologies)

Code (in `src/pso_wifi_placement.py`)	Theoretical principle	Where discussed
`X = np.clip(X, lower, upper); V[hit]=0`	**Constraint repair** for the boundary (box) constraint; absorbing walls	Part 1 (constraint handling); Part 3 Q
`breach += max(0, d_min-‖a_j-a_k‖)**2`	**Penalty method** for the inequality (separation) constraint	Part 1; Part 3 "constraint" trap
`pbest/pbest_fit` update on `improved`	Elitist personal memory: never forget your best (monotone `pbest`)	Part 1 (convergence)
`if pbest_fit[best] > gbest_fit: gbest=…`	Global-best elitism → guarantees the returned `history` is monotone non-decreasing	Part 1 (convergence definition)
`_diversity()` = mean distance to centroid	Quantitative diversity metric to *detect* premature convergence	Part 1 (premature convergence); `diversity.png`
`_stagnation_iter()`	Stopping-criterion analysis: last iteration with > `eps` improvement	Part 3 (stopping criteria)
`random_search(... n_evals ...)` and the `assert n_fitness_calls == n_evals`	**Fair baseline**: identical fitness-evaluation budget (not identical iterations)	Part 3 (fair comparison trap)
`grid_search()` with `C(G,K) ≤ budget`	Honest grid baseline; its inability to scale = live curse-of-dimensionality demo	Part 3
`stats.wilcoxon(pso_best, rand_best, alternative='greater')`	Non-parametric paired significance test over independent runs	Part 3 (statistical significance trap)
multi-seed `run_experiments` (mean ± std)	Metaheuristics are stochastic → report distributions, not a single lucky run	Part 3

1.2 5.2 Biological-analogy annotations (already inline in the code)

- **Particle** = a bird in a flock hunting the best feeding spot; its position is a whole candidate AP layout.
- **`pbest`** = the bird's private memory / nostalgia for the best spot it found.
- **`gbest`** = the socially broadcast best spot found by any bird.
- **inertia `w`** = the bird's momentum; **`r1,r2`** = the random flutter of wings.
- **velocity clamp** = a physical speed limit so a bird cannot teleport across the whole hall in one wingbeat.

1.3 5.3 Three high-probability "pointing-at-the-screen" questions

1.3.1 Q1 — "On the velocity line, what happens if the inertia weight `w` goes to 0? What if it stays at 0.9 the whole run?"

Answer. `w` scales the particle's *momentum* (its previous velocity).

- **`w → 0`:** the velocity loses all memory of prior motion; each step becomes a pure random-weighted pull toward `pbest`/`gbest`. The swarm contracts onto the current global best almost immediately — strong exploitation, but **premature convergence**: if `gbest` is a mediocre local optimum, the swarm is trapped there with no momentum to carry it out.
- **`w = 0.9` fixed:** the opposite failure — particles keep barrelling past good regions (over-exploration), the swarm never settles, and final-iteration accuracy is poor.
- **Why my schedule:** I decay `w` linearly $0.9 \to 0.4$ so the swarm explores broadly early and refines late — the standard, stable Shi–Eberhart (1998) regime. The `diversity.png` curve is the empirical proof this transition happens.

1.3.2 Q2 — "Delete the `np.clip(V, -v_max, v_max)` line — or set `v_max` huge. What breaks?"

Answer. That line is the **velocity clamp**, the classic PSO stability mechanism. Without it, the term $w\mathbf{v} + c_1 r_1(\cdot) + c_2 r_2(\cdot)$ can grow without bound when a particle is far from `pbest`/`gbest`: velocities **explode**, positions overshoot the floor every step, and the swarm diverges instead of converging (the well-documented "swarm explosion"). The subsequent position clamp would then just pin particles to the walls with zeroed velocity, destroying the search. `v_max` is set to 20 % of each axis range so a particle moves in meaningful sub-floor steps. (The constriction-factor variant of Clerc & Kennedy 2002 is the alternative fix; I use inertia + `v_max` to match the course slide's update rule.)

1.3.3 Q3 — "Why `np.maximum(dist, 1.0)` before the `log10`? Remove it — what happens?"

Answer. The radio model is **log-distance path loss** referenced to $d_0 = 1$ m: $\text{PL} = L_0 + 10n \log_{10}(d/d_0)$. Two reasons for the floor:

1. **Numerics:** if an AP sits exactly on a room, $d = 0$ and $\log_{10} 0 = -\infty$, giving RSSI $= +\infty$ and `NaN` fitness that poisons the whole run.
2. **Physics:** the log-distance model is only valid in the far field ($d \geq d_0$); below 1 m it is meaningless. Flooring at 1 m keeps every evaluation inside the model's domain. Removing it makes "stack an AP on the busiest room" look infinitely good — a degenerate optimum the optimizer would happily exploit.

1.4 5.4 What the final printout proves (for the viva)

Running `./run.sh swarm` prints, in order:

- the **best solution vector** (the K AP coordinates),
- its **fitness** (weighted coverage %) and **hard room coverage %**,
- the **iteration at which convergence stagnated**,
- a **side-by-side matrix** (PSO vs Random vs Grid) at the *identical* budget,
- the **Wilcoxon** statistic and p-value with a significance verdict.

This is the complete evidentiary chain: *what* was found, *how good* it is, *when* it converged, and *whether the advantage over the baseline is real*.

---

2 Part 5B — Code Defense: the decision-making code

Companion to `src/rl_water_tank.py` (the MDP, Value Iteration, Q-learning, certainty equivalence) and `src/rl_experiments.py` (the experiments that produce every number and figure). Same job as above: map the code onto the theory, then pre-load the answers to the questions an examiner asks while pointing at a line.

2.1 5.5 Code → theory mapping

Code	Theoretical principle	Where discussed
`value_iteration()`: `Q = R + gamma*(P@V).reshape(S,A)` then `V_new = _masked(Q,avail).max(axis=1)`	The **Bellman optimality operator** $(\mathcal{T}V)(s) = \max_{a \in \mathcal{A}(s)} \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V(s')\right]$, applied synchronously. The `max` is what makes it *optimality* rather than *expectation*	Part 1 (Bellman equations)
`residuals.append(delta)`, 'delta =	V_new - V	_inf'
`if delta < tol: break`, `tol=1e-9`	Stopping rule justified by the standard bound $\|V_k - V^*\|_\infty \le \gamma\varepsilon/(1-\gamma)$ — not picked by feel	Part 1 (VI error bound)
`policy_evaluation()`: `spla.spsolve(sp.eye(S) - gamma*P_pi, R_pi)`	The **Bellman expectation equation** for a *fixed* policy, $V^\pi = R_\pi + \gamma P_\pi V^\pi$, is **linear** — so we solve $(I - \gamma P_\pi)V^\pi = R_\pi$ exactly rather than iterating it. $(I - \gamma P_\pi)$ is invertible because $\rho(\gamma P_\pi) \le \gamma < 1$	Part 1; §5.7 Q6
`build_model()` → `R[s,a] = -(pump_cost + c_ov*overflow + c_short*exp_unmet)` where `exp_unmet = p_demand @ max(d - after_pump, 0)`	**Expected reward** $R(s,a) = \mathbb{E}_w[r(s,a,w)]$, $w = (D, g')$. The reward is *not* a function of (s,a,s') — once the tank hits zero you cannot recover D from s' — so we are in the disturbance form of an MDP. VI only ever needs the expectation, and the operator is still a contraction	Part 1 (MDP formalism)
`simulate_hour()` → `unmet = max(0, demand - after_pump); reward = -(...)`	The **realised** reward, from a *drawn* demand. Q-learning must consume this. Feeding it $\mathbb{E}[U]$ would leak the demand distribution into the "model-free" agent and collapse the entire point of the assignment	Part 1 (model-free)
`step()` — the only method Q-learning is allowed to call	The **sampling oracle**. No probability ever leaves this function. This narrow doorway *is* the model-based / model-free distinction, made mechanical rather than rhetorical	Part 1
`build_model()` returning a sparse `csr_matrix` of shape `(S*A, S)`	The explicit $P(s' \mid s,a)$ that makes VI *model-based* in the most literal sense: the sum over s' is only computable because somebody handed us P. Each row has ≤ 14 non-zeros; a dense $S \times A \times S$ array would be 99.9% zeros	Part 1
`q_learning()`: `td_target = r + gamma*Q[s_next].max()`; `Q[s,a] += alpha*(td_target - Q[s,a])`	The **temporal-difference update** $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$. The `.max()` is the **off-policy** part: we bootstrap off the *best* next action even though ε-greedy may not take it. That is precisely what lets the agent converge to Q^* while behaving sub-optimally throughout	Part 1 (Q-learning); §5.7 Q4

Code	Theoretical principle	Where discussed
`alpha = 1.0/(1.0 + visits[s,a])**qcfg.alpha_omega, alpha_omega=0.7`	**Robbins–Monro** conditions: $\sum_n \alpha_n = \infty$, $\sum_n \alpha_n^2 < \infty$. $\omega \in (0.5, 1]$ delivers both. A *constant* α satisfies neither and provably converges only to a *neighbourhood* of Q^* — we do not just assert this, the sweep shows constant $\alpha = 0.1$ stalling at **50.43%** regret where the polynomial schedule reaches **25.07%**	Part 1 (stochastic approximation)
`eps = max(eps_end, eps_start - (…)*ep/anneal_over)`	ε-greedy **exploration–exploitation** schedule: explore early, exploit late. The mirror of PSO's inertia decay in §5.1	Part 1 (explore/exploit)
`if qcfg.exploring_starts: s = rng.integers(S)`	**Exploring starts**: buys the coverage that Q-learning's convergence proof simply *assumes* (every (s, a) visited infinitely often). Turning it off costs **76.73%** regret against 25.07% — the single most consequential knob in the sweep	Part 1 (convergence conditions)
`_build_availability()` → boolean `(S,3)` mask; `Q[~avail] = -inf`; `_masked()`	**State-dependent action sets** $\mathcal{A}(s)$. `PUMP_GRID` needs the grid, `PUMP_GEN` needs diesel. 3432 legal (s, a) of 4752. Not realism-decoration — see §5.7 Q5	Part 4 §B1.2
`certainty_equivalence()`: `P_hat = counts/n`, `R_hat = reward_sums/n`, then `value_iteration(P_hat, …)`	**Certainty equivalence** / model-based RL: form the maximum-likelihood MDP from experience and plan on it as if it were true. Run on the *same samples* Q-learning saw, at the same instant. It is the arm that could have sunk the report's thesis, and it very nearly did — **1.43%** regret against Q-learning's **15.13%**	Part 1 (model-based RL)
`worst = -(cost_gen + c_ov*pump_rate + c_short*max_demand)` for unvisited (s, a), plus a self-loop	Pessimism under uncertainty: an action never tried cannot look attractive by accident. The alternative (optimism) would make CE-VI fall in love with unexplored actions	Part 1
`policy_myopic() = _masked(R, avail).argmax(axis=1)`	Value iteration at $\gamma = 0$: the **formally correct greedy policy**. Not a strawman — if it landed near the optimum, the problem would have no long-horizon structure and the whole assignment would be vacuous. It loses **120.62%**	Part 4 §B1.3
`tune_caretaker()` grid-searching both thresholds	**Fair baseline**: we report the *best* version of the reactive rule, not a hand-crippled one. Beating a crippled baseline proves nothing	Part 3 (fair-comparison trap)

Code	Theoretical principle	Where discussed
`score_policy()` / `regret_percent()` — exact V^π under the *true* P	Every policy in the report is ranked by **exact expected discounted return**, never by whichever run drew a luckier rollout. Monte-Carlo noise is removed from the comparison entirely	Part 3 (statistical rigour)
`action_optimality()`: `(Q*.max(1) - Q*[range, policy]) <= tol`	Deliberately *not* a label match against π^*. Many states are near-ties; a raw `argmax` comparison would report loud "disagreement" where the two choices differ by 0.001. We ask the question that matters: is the chosen action *as good as* the best available one?	Part 3
`e4_mismatch()`: `mdp.build_model(outage_scale=k)` then score in the *true* model	**Model mis-specification**. `outage_scale` multiplies p_{fail} only — "twice as often" and "twice as long" are different errors with different consequences, so we vary exactly one	Part 4 §B1.4
multi-seed `seeds=[1..5]`, mean ± std everywhere	RL is stochastic → report distributions, not a single lucky run. Same discipline as the swarm half	Part 3

2.2 5.6 The one invariant the whole report rests on

`HallWaterMDP` has two faces, and keeping them honest is the entire trick:

- **`build_model()`** hands out P and R. This is the *only* thing Value Iteration touches. It averages over the randomness.
- **`step()`** hands out one sampled (s', r). This is the *only* thing Q-learning touches. It samples the randomness.

Both call the same `_physics()` helper, so the two faces cannot disagree about what a pump-hour *does* — only about whether they average over the disturbance or draw it. And `tests/test_rl_water_tank.py` Monte-Carlo-estimates P and R back out of `step()` and asserts they match `build_model()` within sampling error. That parity test is the gate: if it fails, every VI-vs-Q-learning number in the report is meaningless, because the two algorithms would be solving different problems.

2.3 5.7 Three high-probability "pointing-at-the-screen" questions

2.3.1 Q4 — "In `q_learning`, the TD target uses `Q[s_next].max()`. Change it to `Q[s_next][a_next]`, where `a_next` is the ε-greedy action you will actually take next. What algorithm have you just written, and what changes?"

Answer. That is **SARSA** — the on-policy TD control algorithm (Rummery & Niranjan 1994). The change is one character wide and it changes what the algorithm converges to.

- **Q-learning is off-policy.** The `max` bootstraps off the *greedy* action regardless of what the behaviour policy actually does next. The target is therefore an estimate of Q^*, and under the Robbins–Monro conditions plus infinite visitation, $Q \to Q^*$ — **the optimal action-values, even though the agent never once behaves optimally during training**. That decoupling is the whole selling point.

- **SARSA is on-policy.** The target $r + \gamma Q(s', a')$ uses the action the ε-greedy policy *will actually take*, so it converges to Q^{π_ε} — the value of the **exploring** policy, not the optimal one. It reaches Q^* only if you also anneal $\varepsilon \to 0$ (a GLIE schedule).
- **What it would look like here.** SARSA's values are pessimistic about states where exploration is expensive, because it *charges itself* for the random actions it is going to take. In this MDP the expensive random action is burning diesel — with ε floored at 0.05 the agent randomly torches its ration in 5% of steps, and SARSA, unlike Q-learning, prices that in. So the SARSA policy would be more conservative about states near the edge of a shortage. The classic cliff-walking result in miniature: SARSA learns the safe path, Q-learning learns the optimal one.
- **Why we chose Q-learning.** We are comparing against Value Iteration, which returns π^*. Comparing π^* against π^ε would be comparing two different optima and the regret numbers would be uninterpretable. Q-learning is the model-free algorithm that targets the *same* fixed point VI does, which is the only reason the head-to-head is fair.

2.3.2 Q5 — "Delete the line `Q[~avail] = -np.inf`. What breaks?"

Answer. Three things break, and — the interesting part — the *optimal value* is not one of them.

1. **The agent can now select actions that do nothing.** With the mask gone, the ε-greedy `argmax` can pick `PUMP_GRID` in a state where the grid is down, or `PUMP_GEN` with an empty diesel drum. Follow it into `_physics()`: neither branch matches, so it falls through to `else: inflow, pump_cost = 0, 0.0`. The action is a **bit-for-bit no-op — an exact duplicate of `IDLE`**.
2. **A third of exploration is wasted.** Exploration would sample uniformly from all three actions instead of from $\mathcal{A}(s)$. In the **792 outage states**, `PUMP_GRID` is a duplicate of `IDLE`, so a third of the exploratory steps taken there teach us nothing we were not already learning. Same story in the 528 zero-diesel states for `PUMP_GEN`. That is why we pre-compute `legal = [np.flatnonzero(avail[s]) …]` and sample from *that*.
3. **Every `argmax` in those 792 states becomes a coin flip.** $Q(s, \text{idle})$ and $Q(s, \text{pump/grid})$ converge to the *identical* value, so the argmax between them is decided by floating-point noise. Any policy-agreement metric between VI and Q-learning would then be measuring **how the two implementations happen to break ties**, not how good their policies are. The reported agreement number would be garbage, and it would be garbage in a way that looks like a real result.

But V^* is unchanged. Adding a duplicate of an action already in $\mathcal{A}(s)$ cannot change $\max_a Q(s, a)$ — the max of a set is unaffected by repeating an element of it. So Value Iteration would still compute exactly the same optimal value function and (up to tie-breaking) the same policy. The mask is not there to protect the mathematics. It is there to protect the **experiment**: it keeps the action set honest so that exploration is not diluted and so that policy comparisons measure policies. This is also why `action_optimality()` scores "is the chosen action as good as the best one?" instead of "does the label match π^*?" — the same near-tie problem, one level up.

2.3.3 Q6 — "Why is `policy_evaluation` a linear `spsolve`, but `value_iteration` a loop? Solve them both."

Answer. Because one of them is linear and the other one is not, and the difference is exactly the `max`.

- **Fixed policy.** With π fixed, every state has exactly one action, so $R_\pi \in \mathbb{R}^S$ and $P_\pi \in \mathbb{R}^{S \times S}$ are just matrices you can slice out (`rows = arange(S)*A + policy`). The Bellman *expectation* equation is
$$V^\pi = R_\pi + \gamma P_\pi V^\pi \quad \Longleftrightarrow \quad (I - \gamma P_\pi) V^\pi = R_\pi,$$
which is **affine in** V — an $S \times S$ linear system. P_π is row-stochastic so $\rho(P_\pi) = 1$, hence $\rho(\gamma P_\pi) \le \gamma < 1$, hence $(I - \gamma P_\pi)$ is non-singular and the solution is *unique*. One sparse LU solve returns it **exactly** — no tolerance, no iteration count, no truncation error.
- **Optimal policy.** The Bellman *optimality* equation is
$$V^*(s) = \max_{a \in \mathcal{A}(s)} \Big[R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^*(s') \Big].$$

The max is piecewise-linear and convex, but it is **not linear**. There is no matrix to invert. The only handle we have is that the operator $\mathcal{T}$ is a γ-contraction in $\|\cdot\|_\infty$, so Banach's fixed-point theorem guarantees a unique V^* and guarantees that iterating $\mathcal{T}$ converges to it geometrically. Geometrically, not finitely: at $\gamma = 0.99$ it takes **2273 sweeps** to drive the residual under 10^{-9} (0.23 s), and the measured decay rate is **0.9900** — exactly γ, as the theory says.

- **The two together are policy iteration.** Alternating "solve exactly for V^π" with "take the greedy policy w.r.t. it" is Howard's policy-iteration algorithm. We have both halves in the file; we run VI because it needs no policy-improvement bookkeeping and 0.23 s is not a runtime we need to optimise.
- **Why the exactness matters to the report.** `policy_evaluation` is the **scoring function for every policy we report** — VI's, Q-learning's, certainty-equivalence's, the tuned caretaker's. Because it is a solve and not a rollout, the ranking between policies carries **zero Monte-Carlo noise**: when we say Q-learning lands at 15.13% regret and CE-VI at 1.43%, that gap is a property of the policies, not of the random seeds we happened to draw.

2.4 5.8 What the RL printout proves (for the viva)

Running `./run.sh rl` prints, in order:

- **The MDP, stated honestly.** $|\mathcal{S}| = 1584$ (level 11 × hour 24 × grid 2 × diesel 3), 3432 legal (s, a) of 4752, the sparsity of P, and $\gamma = 0.99 \Rightarrow$ a 100-hour effective horizon. The examiner can check the arithmetic on the spot.
- **E1 — Value Iteration does what the theory promised.** 2273 sweeps, 0.23 s, empirical contraction rate **0.9900** against a predicted $\gamma = 0.99$; and a cross-check that $\|V^* - V^{\pi^*}\|_\infty \approx 0$, i.e. the iterative operator and the exact linear solve agree. $V^* = -163.637$. This is the number everything else is measured against.
- **E2 — the problem is not vacuous.** The formally correct greedy policy loses **120.62%**; the *tuned* reactive threshold rule — same actions as the optimum, both thresholds grid-searched — still loses **14.51%**. Lookahead is doing real work.
- **E3 — what experience is worth.** After **720,000 environment steps** (82 simulated years of hall operation), Q-learning reaches **15.13% ± 2.36** regret. Certainty-equivalence VI, planning on a model estimated from *exactly the same samples at the same instant*, reaches **1.43% ± 0.23**. Same data, two ways of spending it, an order of magnitude apart.
- **E4 — how wrong a model can afford to be.** A planner who believes outages are four times rarer than they are loses **2.66%**. One who is 25% off loses **0.11%**. One who believes the grid never fails loses **21.39%**. So a *roughly* right prior model beats 82 years of experience, and a *badly* wrong one does not. That is the finding, and it is not the one we expected.
- **E5 — which knobs matter.** Exploring starts OFF: **76.73%** regret, against **25.07%** with them ON. Constant $\alpha = 0.1$: **50.43%**, against **25.07%** for the polynomial Robbins–Monro schedule. Both ablations confirm a theoretical prediction rather than merely tuning a number.
- **E6 — the anticipation witness.** At 14:00, tank 4/10, grid up: pumping costs **0.97 more** this hour than idling, yet Q^* says its advantage is **+4.69**. So **+5.66 of value comes purely from the future**. That single line is the entire argument for lookahead, in one state, with a number attached.

Together: *the model is stated, the exact optimum is computed and verified against theory, the greedy and reactive alternatives are shown to fail, the model-free agent is given a fair and generous budget, the honest baseline that could have refuted us is run anyway,* and *the one thing lookahead buys is exhibited in a single state.*